



# LABORATORIO 2

## MÉTODOS DE ORDENAMIENTO Y BÚSQUEDA

*Java: versión Swing y versión estructurada para la web*

### Contenido

1. Introducción
2. Dos versiones del código: Swing y Web
3. Lectura y validación del arreglo
4. Método Burbuja
5. Burbuja con señal
6. Baraja (inserción)
7. Sacudida
8. Selección directa
9. Shell
10. Búsqueda binaria
11. Diferencias entre Java Swing y código estructurado para la web
12. POO aplicada
13. Relación con el servidor del Laboratorio 2
14. Comparación general
15. Ejemplo integrado
16. Conclusiones

## 1. Introducción

Este documento reúne las clases de ordenamiento y búsqueda del Laboratorio 2 en dos presentaciones. La primera corresponde a la versión Java Swing usada para ejecutar cada algoritmo de forma independiente mediante JOptionPane, JTextArea y cuadros de diálogo. La segunda corresponde a una estructura preparada para trabajar con la página web: no contiene JOptionPane ni main, recibe los datos desde el servidor y devuelve resultados que pueden convertirse a JSON.

## 2. Dos versiones del código: Swing y Web

| Característica      | Java Swing                                     | Estructurado para la web                                     |
|---------------------|------------------------------------------------|--------------------------------------------------------------|
| Entrada             | JOptionPane solicita tamaño y números.         | El navegador envía tam y elementos al servidor.              |
| Salida              | JTextArea y JOptionPane muestran el resultado. | La clase devuelve int[] o int; el servidor forma JSON.       |
| main                | Cada archivo puede ejecutarse por separado.    | No necesita main.                                            |
| Dependencia gráfica | import javax.swing.*;                          | No utiliza javax.swing.                                      |
| Uso                 | Programa de escritorio / práctica individual.  | Clase reutilizable desde S_Burbuja, S_Shell, S_Binaria, etc. |

**Importante:** La lógica de ordenamiento no cambia. Lo que cambia es la forma de recibir y devolver los datos. En Swing el propio algoritmo conversa con el usuario; en la web esa responsabilidad se divide entre HTML/JavaScript, servidor y clase Java.

## 3. Lectura y validación del arreglo

### 3.1 En Java Swing

El mismo archivo solicita el tamaño y cada número.

```
int tam = Integer.parseInt(
    JOptionPane.showInputDialog("Ingrese el tamaño del arreglo")
);

int vector[] = new int[tam];

for (int i = 0; i < tam; i++) {
    vector[i] = Integer.parseInt(
        JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + "]")
    );
}
```

### 3.2 En la estructura web

La clase del algoritmo ya no pide datos. Recibe un arreglo construido a partir de los parámetros que llegan al servidor. Para proteger el arreglo original se trabaja sobre una copia.

```
public static int[] ordenar(int[] vector, int tam) {
    int[] resultado = vector.clone();

    // aquí se ejecuta el algoritmo
    // sobre la copia
}
```

```
    return resultado;
}
```

## 4. Método Burbuja

Compara elementos vecinos y los intercambia cuando están en orden incorrecto.

### Lógica paso a paso

1. Realizar una pasada por el arreglo.
2. Comparar `vector[i]` con `vector[i + 1]`.
3. Intercambiar si el valor izquierdo es mayor.
4. Repetir hasta obtener el arreglo ordenado.

Ejemplo: 8, 3, 7, 1, 5  $\rightarrow$  1, 3, 5, 7, 8

### Código Java Swing

Esta es la versión ejecutable con `javax.swing`. El mismo archivo solicita los datos, ejecuta el algoritmo y muestra el arreglo desordenado y el arreglo ordenado.

```
package Lab_2;

import javax.swing.*;

public class BurbujaProfe {

    public static void Burbuja_Sort(int vector[], int tam) {
        int temp;

        for (int p = 1; p <= tam; p++) {
            for (int i = 0; i < tam - 1; i++) {
                if (vector[i] > vector[i + 1]) {
                    temp = vector[i];
                    vector[i] = vector[i + 1];
                    vector[i + 1] = temp;
                }
            }
        }
    }

    public static void main(String args[]) {
        int tam = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el tamaño del arreglo"));

        int vector[] = new int[tam];
        String sal = "Arreglo desordenado \n";

        for (int i = 0; i < tam; i++) {
            vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + "]"));

            for (int k = i - 1; k >= 0; k--) {
                while (vector[k] == vector[i]) {
                    vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Elemento " + i + " repetido\nIngrese otro número"));
                }
            }

            sal += vector[i] + " ";
        }

        sal += "\nArreglo ordenado \n";

        Burbuja_Sort(vector, tam);

        for (int i = 0; i < tam; i++) {
            sal += vector[i] + " ";
        }

        JTextArea imp = new JTextArea(sal);
        imp.setEditable(false);
    }
}
```

```
JOptionPane.showMessageDialog(null,imp,"Algoritmo ejecutado: BURBUJA",JOptionPane.INFORMATION_MESSAGE);

System.exit(0);
}
}
```

## Código estructurado para la web

Esta versión conserva el algoritmo, elimina javax.swing y el método main, recibe el arreglo como parámetro y devuelve una copia ordenada. Está pensada para ser invocada por la clase S\_ correspondiente del servidor.

```
package Lab_2.Ordenamiento;

public class Burbuja {

    public static int[] ordenar(int[] vector, int tam) {
        int[] resultado = vector.clone();
        Burbuja_Sort(resultado, tam);
        return resultado;
    }

    public static void Burbuja_Sort(int[] vector, int tam) {
        int temp;

        for (int p = 1; p <= tam; p++) {
            for (int i = 0; i < tam - 1; i++) {
                if (vector[i] > vector[i + 1]) {
                    temp = vector[i];
                    vector[i] = vector[i + 1];
                    vector[i + 1] = temp;
                }
            }
        }
    }
}
```

## Qué cambió para usarlo en la web

- Se eliminó import javax.swing.\*.
- Se eliminó JOptionPane y JTextArea.
- Se eliminó main porque el servidor es quien llama a la clase.
- El arreglo se recibe como parámetro.
- Se clona el arreglo para conservar el original.
- El resultado se devuelve como int[] para que la capa servidor lo envíe a JavaScript.

## 5. Burbuja con señal

Agrega una bandera que permite detener el algoritmo cuando una pasada completa no realiza intercambios.

### Lógica paso a paso

5. Inicializar la bandera.
6. Comparar pares vecinos.
7. Intercambiar cuando corresponda.
8. Marcar que hubo cambio.
9. Detenerse si una pasada termina sin cambios.

Ejemplo: 8, 3, 7, 1, 5 → 1, 3, 5, 7, 8

## Código Java Swing

Esta es la versión ejecutable con javax.swing. El mismo archivo solicita los datos, ejecuta el algoritmo y muestra el arreglo desordenado y el arreglo ordenado.

```
package Lab_2;

import javax.swing.*;

public class BurbujaSProfe {

    public static void Burbuja_Senal(int vector[], int tam) {
        int i = 1, temp;
        boolean band = false;

        while ((i < tam) && (band == false)) {
            band = true;

            for (int j = 0; j < tam - 1; j++) {
                if (vector[j] > vector[j + 1]) {
                    temp = vector[j];
                    vector[j] = vector[j + 1];
                    vector[j + 1] = temp;

                    band = false;
                }
            }

            i++;
        }
    }

    public static void main(String args[]) {
        int tam = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el tamaño del arreglo"));

        int vector[] = new int[tam];
        String sal = "Arreglo desordenado \n";

        for (int i = 0; i < tam; i++) {
            vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + "]"));

            for (int k = i - 1; k >= 0; k--) {
                while (vector[k] == vector[i]) {
                    vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Elemento " + i + " repetido\nIngrese otro número"));

                    k = i - 1;
                }
            }

            sal += vector[i] + " ";
        }

        sal += "\nArreglo ordenado \n";

        Burbuja_Senal(vector, tam);

        for (int i = 0; i < tam; i++) {
            sal += vector[i] + " ";
        }

        JTextArea imp = new JTextArea(sal);
        imp.setEditable(false);

        JOptionPane.showMessageDialog(null, imp, "Algoritmo ejecutado: BURBUJA SEÑAL", JOptionPane.INFORMATION_MESSAGE);

        System.exit(0);
    }
}
```

## Código estructurado para la web

Esta versión conserva el algoritmo, elimina javax.swing y el método main, recibe el arreglo como parámetro y devuelve una copia ordenada. Está pensada para ser invocada por la clase S\_ correspondiente del servidor.

```
package Lab_2.Ordenamiento;

public class Burbuja_S {
```

```
public static int[] ordenar(int[] vector, int tam) {
    int[] resultado = vector.clone();
    Burbuja_Senal(resultado, tam);
    return resultado;
}

public static void Burbuja_Senal(int[] vector, int tam) {
    int i = 1;
    int temp;
    boolean band = false;

    while ((i < tam) && (band == false)) {
        band = true;

        for (int j = 0; j < tam - 1; j++) {
            if (vector[j] > vector[j + 1]) {
                temp = vector[j];
                vector[j] = vector[j + 1];
                vector[j + 1] = temp;
                band = false;
            }
        }

        i++;
    }
}
```

## Qué cambió para usarlo en la web

- Se eliminó import javax.swing.\*.
- Se eliminó JOptionPane y JTextArea.
- Se eliminó main porque el servidor es quien llama a la clase.
- El arreglo se recibe como parámetro.
- Se clona el arreglo para conservar el original.
- El resultado se devuelve como int[] para que la capa servidor lo envíe a JavaScript.

## 6. Baraja (inserción)

Inserta cada elemento en su posición dentro de la parte izquierda del arreglo que ya está ordenada.

### Lógica paso a paso

10. Tomar el elemento actual como auxiliar.
11. Retroceder por la zona ordenada.
12. Desplazar a la derecha los valores mayores.
13. Insertar el auxiliar en el espacio disponible.

Ejemplo: 8, 3, 7, 1, 5 → 1, 3, 5, 7, 8

### Código Java Swing

Esta es la versión ejecutable con javax.swing. El mismo archivo solicita los datos, ejecuta el algoritmo y muestra el arreglo desordenado y el arreglo ordenado.

```
package Lab_2;

import javax.swing.*;

public class BarajaProfe {

    public static void Baraja_Sort(int vector[], int tam) {
        int aux, k;

        for (int i = 1; i < tam; i++) {
            aux = vector[i];
            k = i - 1;
```

```
        while (k >= 0 && aux < vector[k]) {
            vector[k + 1] = vector[k];
            k--;
        }

        vector[k + 1] = aux;
    }
}

public static void main(String args[]) {
    int tam = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el tamaño del arreglo"));

    int vector[] = new int[tam];
    String sal = "Arreglo desordenado \n";

    for (int i = 0; i < tam; i++) {
        vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + " ]"));

        for (int k = i - 1; k >= 0; k--) {
            while (vector[k] == vector[i]) {
                vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Elemento " + i + " repetido\nIngrese otro número"));

                k = i - 1;
            }
        }

        sal += vector[i] + " ";
    }

    sal += "\nArreglo ordenado \n";

    Baraja_Sort(vector, tam);

    for (int i = 0; i < tam; i++) {
        sal += vector[i] + " ";
    }

    JTextArea imp = new JTextArea(sal);
    imp.setEditable(false);

    JOptionPane.showMessageDialog(null, imp, "Algoritmo ejecutado: BARAJA", JOptionPane.INFORMATION_MESSAGE);

    System.exit(0);
}
```

## Código estructurado para la web

Esta versión conserva el algoritmo, elimina javax.swing y el método main, recibe el arreglo como parámetro y devuelve una copia ordenada. Está pensada para ser invocada por la clase S\_ correspondiente del servidor.

```
package Lab_2.Ordenamiento;

public class Baraja {

    public static int[] ordenar(int[] vector, int tam) {
        int[] resultado = vector.clone();
        Baraja_Sort(resultado, tam);
        return resultado;
    }

    public static void Baraja_Sort(int[] vector, int tam) {
        int aux;
        int k;

        for (int i = 1; i < tam; i++) {
            aux = vector[i];
            k = i - 1;

            while (k >= 0 && aux < vector[k]) {
                vector[k + 1] = vector[k];
                k--;
            }

            vector[k + 1] = aux;
        }
    }
}
```

```
}
```

## Qué cambió para usarlo en la web

- Se eliminó `import javax.swing.*`.
- Se eliminó `JOptionPane` y `JTextArea`.
- Se eliminó `main` porque el servidor es quien llama a la clase.
- El arreglo se recibe como parámetro.
- Se clona el arreglo para conservar el original.
- El resultado se devuelve como `int[]` para que la capa servidor lo envíe a JavaScript.

## 7. Sacudida

Recorre el arreglo en ambos sentidos: derecha a izquierda y luego izquierda a derecha.

### Lógica paso a paso

14. Definir límites izquierdo y derecho.
15. Recorrer hacia un extremo e intercambiar.
16. Actualizar el límite.
17. Recorrer en sentido contrario.
18. Repetir hasta que los límites se crucen.

Ejemplo: 8, 3, 7, 1, 5 → 1, 3, 5, 7, 8

### Código Java Swing

Esta es la versión ejecutable con `javax.swing`. El mismo archivo solicita los datos, ejecuta el algoritmo y muestra el arreglo desordenado y el arreglo ordenado.

```
package Lab_2;

import javax.swing.*;

public class SacudidaProfe {

    public static void Shaker_Sort(int vector[], int tam) {

        int izq = 1, der = tam - 1, k = tam - 1, temp;

        while (der >= izq) {

            for (int i = der; i >= izq; i--) {
                if (vector[i - 1] > vector[i]) {
                    temp = vector[i - 1];
                    vector[i - 1] = vector[i];
                    vector[i] = temp;

                    k = i;
                }
            }

            izq = k + 1;

            for (int i = izq; i <= der; i++) {
                if (vector[i - 1] > vector[i]) {
                    temp = vector[i - 1];
                    vector[i - 1] = vector[i];
                    vector[i] = temp;

                    k = i;
                }
            }

            der = k - 1;
        }
    }
}
```



```
public static void main(String args[]) {
    int tam = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el tamaño del arreglo"));

    int vector[] = new int[tam];
    String sal = "Arreglo desordenado \n";

    for (int i = 0; i < tam; i++) {
        vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + "]"));

        for (int k = i - 1; k >= 0; k--) {
            while (vector[k] == vector[i]) {
                vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Elemento " + i + " repetido\nIngrese otro número"));

                k = i - 1;
            }
        }

        sal += vector[i] + " ";
    }

    sal += "\nArreglo ordenado \n";

    Shaker_Sort(vector, tam);

    for (int i = 0; i < tam; i++) {
        sal += vector[i] + " ";
    }

    JTextArea imp = new JTextArea(sal);
    imp.setEditable(false);

    JOptionPane.showMessageDialog(null, imp, "Algoritmo ejecutado: SACUDIDA", JOptionPane.INFORMATION_MESSAGE);

    System.exit(0);
}
```

## Código estructurado para la web

Esta versión conserva el algoritmo, elimina javax.swing y el método main, recibe el arreglo como parámetro y devuelve una copia ordenada. Está pensada para ser invocada por la clase S\_ correspondiente del servidor.

```
package Lab_2.Ordenamiento;

public class Sacudida {

    public static int[] ordenar(int[] vector, int tam) {
        int[] resultado = vector.clone();
        Shaker_Sort(resultado, tam);
        return resultado;
    }

    public static void Shaker_Sort(int[] vector, int tam) {
        int izq = 1;
        int der = tam - 1;
        int k = tam - 1;
        int temp;

        while (der >= izq) {
            for (int i = der; i >= izq; i--) {
                if (vector[i - 1] > vector[i]) {
                    temp = vector[i - 1];
                    vector[i - 1] = vector[i];
                    vector[i] = temp;
                    k = i;
                }
            }

            izq = k + 1;

            for (int i = izq; i <= der; i++) {
                if (vector[i - 1] > vector[i]) {
                    temp = vector[i - 1];
                    vector[i - 1] = vector[i];
                    vector[i] = temp;
                    k = i;
                }
            }
        }
    }
}
```

```
    }  
    der = k - 1;  
  }  
}  
}
```

## Qué cambió para usarlo en la web

- Se eliminó import javax.swing.\*.
- Se eliminó JOptionPane y JTextArea.
- Se eliminó main porque el servidor es quien llama a la clase.
- El arreglo se recibe como parámetro.
- Se clona el arreglo para conservar el original.
- El resultado se devuelve como int[] para que la capa servidor lo envíe a JavaScript.

## 8. Selección directa

Busca el menor valor de la zona no ordenada y lo coloca en la posición que corresponde.

### Lógica paso a paso

19. Tomar una posición inicial.
20. Buscar el menor valor desde esa posición.
21. Guardar la posición del menor.
22. Intercambiar con la posición actual.
23. Continuar con la siguiente posición.

Ejemplo: 8, 3, 7, 1, 5 → 1, 3, 5, 7, 8

### Código Java Swing

Esta es la versión ejecutable con javax.swing. El mismo archivo solicita los datos, ejecuta el algoritmo y muestra el arreglo desordenado y el arreglo ordenado.

```
package Lab_2;  
  
import javax.swing.*;  
  
public class SeleccionProfe {  
  
    public static void Seleccion_Sort(int vector[], int tam) {  
        int menor, k;  
  
        for (int i = 0; i < tam; i++) {  
            menor = vector[i];  
            k = i;  
  
            for (int j = i + 1; j < tam; j++) {  
                if (vector[j] < menor) {  
                    menor = vector[j];  
                    k = j;  
                }  
            }  
  
            vector[k] = vector[i];  
            vector[i] = menor;  
        }  
    }  
  
    public static void main(String args[]) {  
        int tam = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el tamaño del arreglo"));  
  
        int vector[] = new int[tam];  
        String sal = "Arreglo desordenado \n";  
  
        for (int i = 0; i < tam; i++) {
```

```
vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + "]"));

for (int k = i - 1; k >= 0; k--) {
    while (vector[k] == vector[i]) {
        vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Elemento " + i + " repetido\nIngrese otro número"));

        k = i - 1;
    }
}

sal += vector[i] + " ";

sal += "\nArreglo ordenado\n";

Seleccion_Sort(vector, tam);

for (int i = 0; i < tam; i++) {
    sal += vector[i] + " ";
}

JTextArea imp = new JTextArea(sal);
imp.setEditable(false);

JOptionPane.showMessageDialog(null, imp, "Algoritmo ejecutado: SELECCIÓN", JOptionPane.INFORMATION_MESSAGE);

System.exit(0);
}
}
```

## Código estructurado para la web

Esta versión conserva el algoritmo, elimina javax.swing y el método main, recibe el arreglo como parámetro y devuelve una copia ordenada. Está pensada para ser invocada por la clase S\_ correspondiente del servidor.

```
package Lab_2.Ordenamiento;

public class Seleccion {

    public static int[] ordenar(int[] vector, int tam) {
        int[] resultado = vector.clone();
        Seleccion_Sort(resultado, tam);
        return resultado;
    }

    public static void Seleccion_Sort(int[] vector, int tam) {
        int menor;
        int k;

        for (int i = 0; i < tam; i++) {
            menor = vector[i];
            k = i;

            for (int j = i + 1; j < tam; j++) {
                if (vector[j] < menor) {
                    menor = vector[j];
                    k = j;
                }
            }

            vector[k] = vector[i];
            vector[i] = menor;
        }
    }
}
```

## Qué cambió para usarlo en la web

- Se eliminó import javax.swing.\*.
- Se eliminó JOptionPane y JTextArea.
- Se eliminó main porque el servidor es quien llama a la clase.
- El arreglo se recibe como parámetro.

- Se clona el arreglo para conservar el original.
- El resultado se devuelve como int[] para que la capa servidor lo envíe a JavaScript.

## 9. Shell

Compara valores separados por saltos que se reducen progresivamente.

### Lógica paso a paso

24. Calcular un salto inicial.
25. Comparar elementos separados por el salto.
26. Intercambiar si corresponde.
27. Reducir el salto.
28. Finalizar cuando el arreglo quede ordenado.

Ejemplo: 8, 3, 7, 1, 5 → 1, 3, 5, 7, 8

### Código Java Swing

Esta es la versión ejecutable con javax.swing. El mismo archivo solicita los datos, ejecuta el algoritmo y muestra el arreglo desordenado y el arreglo ordenado.

```
package Lab_2;

import javax.swing.*;

public class ShellProfe {

    public static void Shell_Sort(int vector[], int tam) {
        int ent = tam + 1, temp, i;
        boolean band;

        while (ent > 0) {
            ent = ent / 2;
            band = true;

            while (band) {
                band = false;
                i = 0;

                while ((i + ent) < tam) {
                    if (vector[i] > vector[i + ent]) {
                        temp = vector[i];
                        vector[i] = vector[i + ent];
                        vector[i + ent] = temp;

                        band = true;
                    }

                    i++;
                }
            }
        }
    }

    public static void main(String args[]) {
        int tam = Integer.parseInt(JOptionPane.showInputDialog("Ingrese el tamaño del arreglo"));

        int vector[] = new int[tam];
        String sal = "Arreglo desordenado \n";

        for (int i = 0; i < tam; i++) {
            vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Ingrese un número en la posición [" + i + "]"));

            for (int k = i - 1; k >= 0; k--) {
                while (vector[k] == vector[i]) {
                    vector[i] = Integer.parseInt(JOptionPane.showInputDialog("Elemento " + i + " repetido\nIngrese otro número"));

                    k = i - 1;
                }
            }

            sal += vector[i] + " ";
        }
    }
}
```

```
sal += "\nArreglo ordenado \n";

Shell_Sort(vector, tam);

for (int i = 0; i < tam; i++) {
    sal += vector[i] + " ";
}

JTextArea imp = new JTextArea(sal);
imp.setEditable(false);

JOptionPane.showMessageDialog(null,imp,"Algoritmo ejecutado: SHELL",JOptionPane.INFORMATION_MESSAGE);

System.exit(0);
}
}
```

## Código estructurado para la web

Esta versión conserva el algoritmo, elimina javax.swing y el método main, recibe el arreglo como parámetro y devuelve una copia ordenada. Está pensada para ser invocada por la clase S\_ correspondiente del servidor.

```
package Lab_2.Ordenamiento;

public class Shell {

    public static int[] ordenar(int[] vector, int tam) {
        int[] resultado = vector.clone();
        Shell_Sort(resultado, tam);
        return resultado;
    }

    public static void Shell_Sort(int[] vector, int tam) {
        int ent = tam + 1;
        int temp;
        int i;
        boolean band;

        while (ent > 0) {
            ent = ent / 2;
            band = true;

            while (band) {
                band = false;
                i = 0;

                while ((i + ent) < tam) {
                    if (vector[i] > vector[i + ent]) {
                        temp = vector[i];
                        vector[i] = vector[i + ent];
                        vector[i + ent] = temp;
                        band = true;
                    }

                    i++;
                }
            }
        }
    }
}
```

## Qué cambió para usarlo en la web

- Se eliminó import javax.swing.\*.
- Se eliminó JOptionPane y JTextArea.
- Se eliminó main porque el servidor es quien llama a la clase.
- El arreglo se recibe como parámetro.
- Se clona el arreglo para conservar el original.
- El resultado se devuelve como int[] para que la capa servidor lo envíe a JavaScript.

## 10. Búsqueda binaria

La búsqueda binaria trabaja sobre un arreglo ordenado. Mantiene los límites inicio y fin, calcula una posición central y descarta la mitad que no puede contener el dato.

29. Definir inicio = 0 y fin = tam - 1.
30. Calcular centro = (inicio + fin) / 2.
31. Si vector[centro] es el número buscado, devolver centro.
32. Si el número buscado es menor, mover fin a centro - 1.
33. Si es mayor, mover inicio a centro + 1.
34. Si el ciclo termina, devolver -1.

### Código Java Swing / POO

La versión disponible usa Swing y además muestra cómo aplicar abstracción, encapsulamiento, herencia y polimorfismo a una clase de búsqueda.

```
package Lab_2;

import javax.swing.*;

/*
 * BinariaPOO.java
 * Un solo archivo y una sola clase pública.
 * Dentro están: clase padre, clase hija, main y métodos de apoyo.
 */

public class BinariaPOO {

    // =====
    // 1. CLASE PADRE
    // =====
    // ABSTRACCIÓN: representa cualquier algoritmo de búsqueda.
    private abstract static class AlgoritmoBusqueda {

        // ENCAPSULAMIENTO:
        // los atributos no se acceden directamente desde afuera.
        private final int[] vector;
        private final int tam;

        // Constructor de la clase padre.
        protected AlgoritmoBusqueda(int[] vector, int tam) {

            // clone() protege el arreglo original.
            this.vector = vector.clone();
            this.tam = tam;
        }

        // Cada clase hija debe definir su propia forma de buscar.
        public abstract int buscar(int numero);

        // Métodos que puede utilizar la clase hija.
        protected final int getTam() {
            return tam;
        }

        protected final int obtener(int posicion) {
            return vector[posicion];
        }

        // Devuelve una copia para no exponer el vector interno.
        public final int[] getVector() {
            return vector.clone();
        }
    }

    // =====
    // 2. CLASE HIJA
    // =====
    // HERENCIA:
    // esta clase hereda de AlgoritmoBusqueda.
    private static class BusquedaBinaria extends AlgoritmoBusqueda {

        // Envía el arreglo y su tamaño al constructor de la clase padre.
        protected BusquedaBinaria(int[] vector, int tam) {
            super(vector, tam);
        }
    }
}
```

```
// POLIMORFISMO:
// esta es la versión Binaria del método buscar().
@Override
public int buscar(int numero) {

    int inicio = 0;
    int fin = getTam() - 1;

    while (inicio <= fin) {

        // Se calcula la posición central.
        int centro = (inicio + fin) / 2;

        // Si encontramos el número.
        if (obtener(centro) == numero) {
            return centro;
        }

        // Si el número buscado es menor,
        // se continúa buscando en la mitad izquierda.
        if (numero < obtener(centro)) {

            fin = centro - 1;

        } else {

            // Si es mayor,
            // se continúa en la mitad derecha.
            inicio = centro + 1;
        }
    }

    // Si no se encuentra el número.
    return -1;
}

// =====
// 3. MAIN
// =====
public static void main(String[] args) {

    // Se solicita el tamaño del arreglo.
    Integer tam = pedirTamano();

    if (tam == null) {
        return;
    }

    // La búsqueda binaria necesita un arreglo ordenado.
    int[] vector = leerVectorOrdenadoSinRepetidos(tam);

    if (vector == null) {
        return;
    }

    // Se solicita el número que se desea buscar.
    Integer numero = pedirEntero(
        "Ingrese el número que desea buscar:"
    );

    if (numero == null) {
        return;
    }

    String sal = "Arreglo ordenado\n";
    sal += arregloATexto(vector);

    sal += "\n\nNúmero buscado: " + numero;

    // POLIMORFISMO:
    // variable de la clase padre con objeto de la clase hija.
    AlgoritmoBusqueda miBusqueda =
        new BusquedaBinaria(vector, tam);

    int posicion = miBusqueda.buscar(numero);

    if (posicion != -1) {

        sal += "\n\nEl número " + numero +
            " fue encontrado.";
    }
}
```

```
sal += "\nPosición: [" + posicion + "]";

} else {

    sal += "\n\nEl número " + numero +
        " no se encuentra en el arreglo.";
}

JTextArea imp = new JTextArea(sal);
imp.setEditable(false);

JOptionPane.showMessageDialog(
    null,
    imp,
    "Algoritmo ejecutado: BÚSQUEDA BINARIA",
    JOptionPane.INFORMATION_MESSAGE
);
}

// =====
// 4. MÉTODOS DE APOYO PARA LAS VENTANAS
// =====

// Pide un tamaño mayor que cero.
private static Integer pedirTamano() {

    while (true) {

        Integer tam = pedirEntero(
            "Ingrese el tamaño del arreglo:"
        );

        if (tam == null) {
            return null;
        }

        if (tam > 0) {
            return tam;
        }

        JOptionPane.showMessageDialog(
            null,
            "El tamaño debe ser mayor que cero.",
            "Dato inválido",
            JOptionPane.WARNING_MESSAGE
        );
    }
}

// Lee un arreglo ordenado de menor a mayor
// y no permite números repetidos.
private static int[] leerVectorOrdenadoSinRepetidos(int tam) {

    int[] vector = new int[tam];

    for (int i = 0; i < tam; i++) {

        while (true) {

            Integer numero = pedirEntero(
                "Ingrese un número en la posición [" + i + "]: "
            );

            if (numero == null) {
                return null;
            }

            // Evita números repetidos.
            if (existeNumero(vector, i, numero)) {

                JOptionPane.showMessageDialog(
                    null,
                    "Elemento " + i +
                    " repetido. Ingrese otro número.",
                    "Número repetido",
                    JOptionPane.WARNING_MESSAGE
                );

            } else if (i > 0 && numero < vector[i - 1]) {

                // La búsqueda binaria necesita que
                // el arreglo esté ordenado.
                JOptionPane.showMessageDialog(
```



```
        null,
        "El arreglo debe ingresarse ordenado de menor a mayor.\n" +
        "Ingrese un número mayor que " + vector[i - 1] + ". ",
        "Arreglo no ordenado",
        JOptionPane.WARNING_MESSAGE
    );

    } else {

        vector[i] = numero;
        break;
    }
}

return vector;
}

// Convierte el texto ingresado en un número entero.
private static Integer pedirEntero(String mensaje) {

    while (true) {

        String entrada =
            JOptionPane.showInputDialog(null, mensaje);

        if (entrada == null) {
            return null;
        }

        try {

            return Integer.parseInt(
                entrada.trim()
            );

        } catch (NumberFormatException e) {

            JOptionPane.showMessageDialog(
                null,
                "Ingrese un número entero válido.",
                "Dato inválido",
                JOptionPane.WARNING_MESSAGE
            );
        }
    }
}

// Revisa las posiciones que ya fueron llenadas.
private static boolean existeNumero(
    int[] vector,
    int hasta,
    int numero) {

    for (int i = 0; i < hasta; i++) {

        if (vector[i] == numero) {
            return true;
        }
    }

    return false;
}

// Convierte el arreglo a texto.
private static String arregloATexto(int[] vector) {

    String texto = "";

    for (int numero : vector) {
        texto += numero + " ";
    }

    return texto;
}
}
```

## Código estructurado para la web

En la versión web no se solicita el número mediante JOptionPane. El servidor recibe valor y tipo de búsqueda y llama al método correspondiente.

```
package Lab_2.Busqueda;

public class Binaria {

    public static int buscar(int[] vector, int tam, int numero) {
        int inicio = 0;
        int fin = tam - 1;

        while (inicio <= fin) {
            int centro = (inicio + fin) / 2;

            if (vector[centro] == numero) {
                return centro;
            }

            if (numero < vector[centro]) {
                fin = centro - 1;
            } else {
                inicio = centro + 1;
            }
        }

        return -1;
    }

    public static int valorEnIndice(int[] vector, int tam, int indice) {
        if (indice < 0 || indice >= tam) {
            throw new IllegalArgumentException("Índice fuera de rango");
        }

        return vector[indice];
    }
}
```

## Búsqueda por número

buscar(...) devuelve el índice del número en el arreglo ordenado o -1 cuando no existe. La capa servidor puede, además, calcular la posición del mismo número en el arreglo original para mostrar ambos resultados.

## Búsqueda por índice

valorEnIndice(...) devuelve el número almacenado en una posición determinada. En la interfaz actual se puede comparar el valor ubicado en el mismo índice del arreglo original y del arreglo ordenado.

## 11. Diferencias entre Java Swing y código estructurado para la web

| Parte              | Swing                     | Web                               | Responsable en la web         |
|--------------------|---------------------------|-----------------------------------|-------------------------------|
| Solicitar tamaño   | JOptionPane               | No se solicita dentro de la clase | HTML + JavaScript             |
| Leer números       | JOptionPane               | Se reciben como int[]             | JavaScript + servidor         |
| Validar HTTP       | No aplica                 | Fuera de la clase de algoritmo    | S_Burbuja / S_Binaria         |
| Ordenar            | Método *_Sort             | Mismo método *_Sort               | Clase de algoritmo            |
| Conservar original | Variable/cadena de salida | vector.clone()                    | Clase de algoritmo / servidor |
| Mostrar resultado  | JTextArea + JOptionPane   | JSON y casillas HTML              | Servidor + JavaScript         |

|          |        |                              |                            |
|----------|--------|------------------------------|----------------------------|
| Ejecutar | main() | configurar(...) del servidor | Servidor.java / Servidor_2 |
|----------|--------|------------------------------|----------------------------|

## 12. POO aplicada

Las clases pueden mantenerse simples con métodos estáticos o evolucionar hacia una jerarquía orientada a objetos. La versión POO de búsqueda muestra cómo una clase padre abstracta define buscar(), mientras la hija BusquedaBinaria sobrescribe ese comportamiento.

| Pilar           | Aplicación                                     | Ejemplo                                    |
|-----------------|------------------------------------------------|--------------------------------------------|
| Encapsulamiento | Protege vector y tamaño.                       | Atributos privados, clone().               |
| Abstracción     | Representa una operación general.              | ordenar() o buscar().                      |
| Herencia        | Una clase especializada reutiliza estructura.  | BusquedaBinaria extends AlgoritmoBusqueda. |
| Polimorfismo    | Se ejecuta la implementación de la clase hija. | @Override buscar().                        |

## 13. Relación con el servidor del Laboratorio 2

En el proyecto web, JavaScript selecciona una ruta distinta para cada método. La capa servidor recibe tam y elementos, convierte los datos a int[], llama a la clase de lógica y devuelve original y ordenado. Las rutas utilizadas por el frontend son las siguientes:

| Método            | Ruta                | Clase servidor |
|-------------------|---------------------|----------------|
| Burbuja           | /lab2/burbuja       | S_Burbuja      |
| Burbuja con señal | /lab2/burbuja-senal | S_Burbuja_S    |
| Baraja            | /lab2/baraja        | S_Baraja       |
| Sacudida          | /lab2/sacudida      | S_Sacudida     |
| Selección         | /lab2/seleccion     | S_Seleccion    |
| Shell             | /lab2/shell         | S_Shell        |
| Búsqueda binaria  | /lab2/binaria       | S_Binaria      |

```
// Ejemplo conceptual dentro de una clase S_
int[] original = vector.clone();
int[] ordenado = Burbuja.ordenar(vector, tam);

// Después la clase S_ construye una respuesta JSON:
// { "ok": true, "original": [...], "ordenado": [...] }
```

## 14. Comparación general

| Algoritmo         | Tipo         | Idea principal               | Uso web             |
|-------------------|--------------|------------------------------|---------------------|
| Burbuja           | Ordenamiento | Comparar vecinos             | /lab2/burbuja       |
| Burbuja con señal | Ordenamiento | Bandera para detenerse antes | /lab2/burbuja-senal |

|           |              |                          |                 |
|-----------|--------------|--------------------------|-----------------|
| Baraja    | Ordenamiento | Inserción progresiva     | /lab2/baraja    |
| Sacudida  | Ordenamiento | Recorrido bidireccional  | /lab2/sacudida  |
| Selección | Ordenamiento | Seleccionar el menor     | /lab2/seleccion |
| Shell     | Ordenamiento | Comparaciones por saltos | /lab2/shell     |
| Binaria   | Búsqueda     | Dividir el rango         | /lab2/binaria   |

## 15. Ejemplo integrado

Supongamos que el usuario confirma tamaño 5 e introduce 8, 3, 7, 1, 5. JavaScript envía los datos al servidor. Si selecciona Burbuja, S\_Burbuja convierte los datos a `int[]`, conserva una copia original y llama a `Burbuja.ordenar(...)`. La clase devuelve 1, 3, 5, 7, 8. El servidor responde JSON y JavaScript muestra ambos arreglos.

Entrada web:  
tam = 5  
elementos = 8,3,7,1,5

Clase Java:  
`int[] ordenado = Burbuja.ordenar(vector, tam);`

Resultado:  
original = [8, 3, 7, 1, 5]  
ordenado = [1, 3, 5, 7, 8]

## 16. Conclusiones

Las versiones Swing y web comparten la misma lógica algorítmica. La diferencia principal está en la responsabilidad de entrada y salida. Swing concentra todo en un programa de escritorio; la solución web separa interfaz, servidor y algoritmo. Esta separación permite reutilizar las clases Java, conservar el arreglo original, devolver resultados al navegador y mantener un solo servidor para todo el Laboratorio 2.